The adoption of a secure coding standard, and not leaving security to the end ensures that security measure is properly integrated into the program throughout the development cycle instead of being a patch add to the end of the life cycle. This ensures that secure coding practices are followed during the creation of the code and can potentially reduce the cost of rewriting code to create a secure program after everything is completed.

The evaluation and assessment of risk and cost benefits of mitigation ensures that high risk, highly probable vulnerabilities are fixed first before fixing minor unlikely bugs. By creating a priority list developers would also have a clear work path to follow to ensure everyone is on the same page and fixing the most important vulnerabilities.

The zero-trust policy assumes that everyone can be a target for an attack. This thought process ensures that no vulnerability or hole in the depth of defense is too small to fix. This also ensures that security measures are not left to the user as the user can make mistakes, fall for scams or phishing attacks, create insecure/predictable passwords, or store passwords insecurely. Assuming that the lowest security clearance user would not lead to admin access assumes that there is no such thing as privilege escalation.

The implementation and recommendations of security policies ensure that everyone who would interact with the program knows and understands security policies of the company to reduce the chance of an employee or an employee's device from being the vector of an attack. This also ensures that developers understand and are reminded of secure coding practices when developing the program.